

MECHATRONICS AND IOT

ASSIGNMENT - 6

Industrial Diesel Generator Monitoring and Maintenance System

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

Industrial Diesel Generator Monitoring and Maintenance System

1. Project Overview

The Industrial Diesel Generator Monitoring and Maintenance System is an IoT-based system designed to continuously monitor the operating condition and performance of a diesel generator. Diesel generators are commonly used as backup power sources in industries, hospitals, commercial buildings, data centers, and other critical facilities.

The proposed system monitors important generator parameters such as temperature, vibration, voltage, current, fuel level, and operating status. An ESP8266 Node MCU can be used as the main controller to collect the sensor data and transmit it through Wi-Fi to a cloud platform.

The collected information can be displayed on a remote dashboard, allowing operators to monitor the generator without being physically present near the equipment. The system also supports preventive maintenance by identifying abnormal operating conditions and providing maintenance alerts.

Main components

- ESP8266 Node MCU
- ACS712 Current Sensor
- Voltage Sensor Module
- DHT11 Sensor
- OLED Display
- Buzzer
- Breadboard

- Jumper Wires

2. Project Statement

Industrial diesel generators must operate reliably because unexpected generator failure can result in production losses, equipment damage, and interruption of critical services.

Traditional generator maintenance often depends on periodic manual inspection. This approach may fail to identify problems that develop between maintenance intervals.

The IoT-based monitoring system is proposed to continuously monitor generator operating parameters, detect abnormal conditions, record historical data, and support preventive maintenance operations.

3. Proposed Solution

The proposed system uses sensors connected to an ESP8266 Node MCU to monitor the condition of the diesel generator.

The sensors collect information such as:

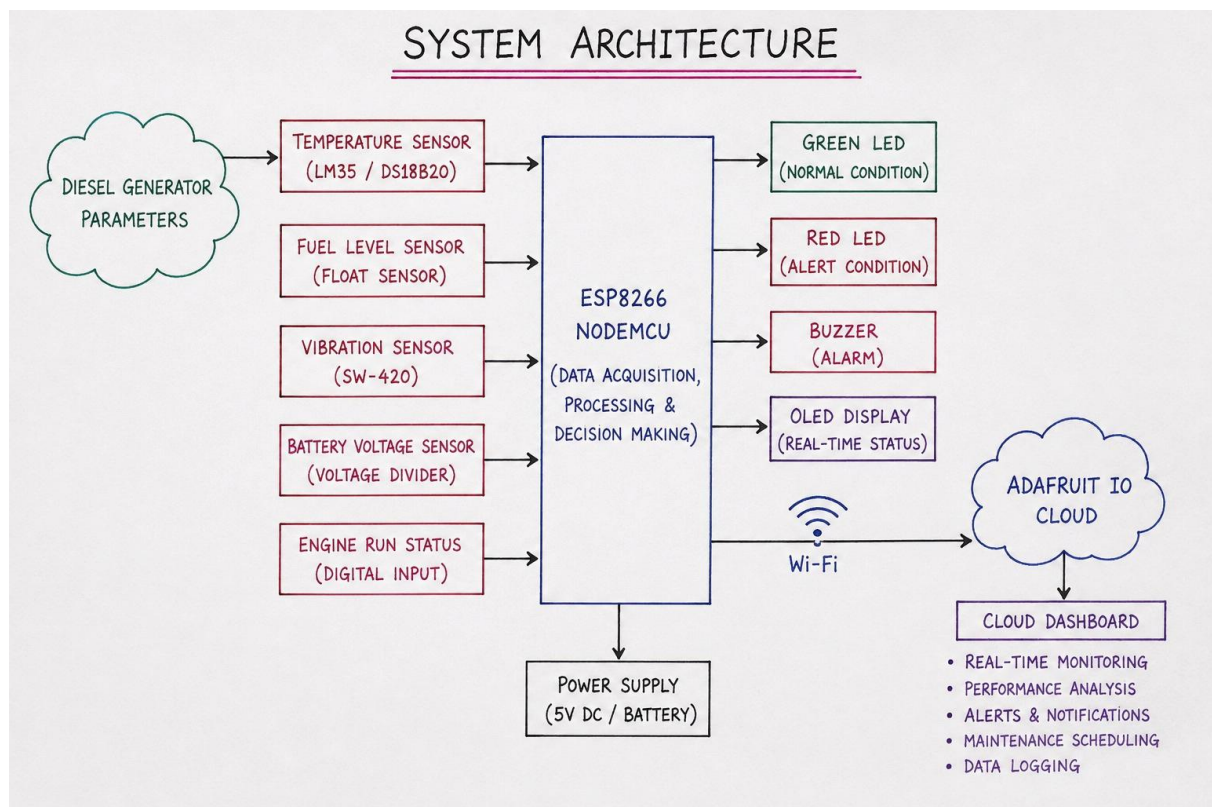
- Engine temperature
- Generator voltage
- Generator current
- Engine vibration
- Fuel level
- Generator operating status

The ESP8266 processes the sensor data and transmits it to a cloud platform using Wi-Fi.

The cloud dashboard provides:

- Real-time generator monitoring
- Historical data storage
- Parameter visualization
- Abnormal-condition detection
- Maintenance notifications
- Preventive-maintenance support

4. System Architecture:



5. Circuit Table:

CIRCUIT TABLE

1. MQ135 GAS SENSOR CONNECTION

Component	Pin	ESP8266 (NodeMCU) Pin	Function
MQ135 Gas Sensor	VCC	VIN / 5V	Power Supply (5V)
	GND	GND	Ground
	AO	A0	Analog Output to ESP8266

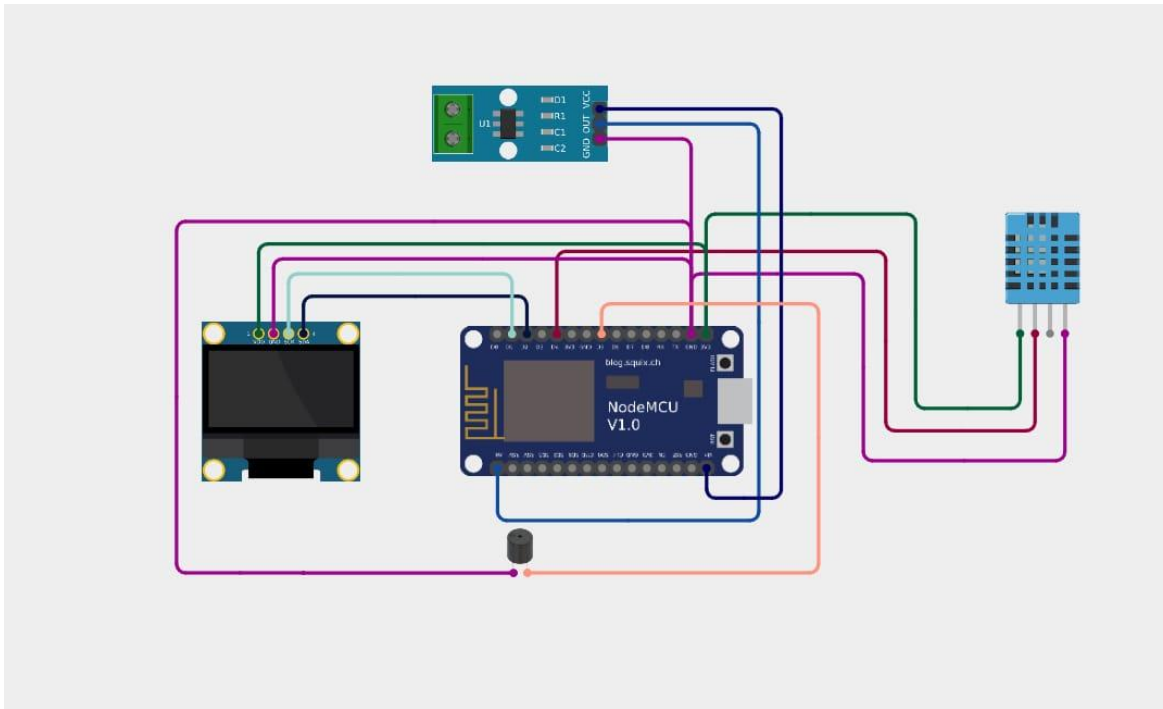
2. LED CONNECTIONS

Component	LED Pin	ESP8266 (NodeMCU) Pin	Other Connection	Function
Green LED	Anode (+)	D5 (GPIO14)	Through 220 Ω Resistor	Indicates Good Air Quality
	Cathode (-)	GND	Direct to GND	–
Red LED	Anode (+)	D6 (GPIO12)	Through 220 Ω Resistor	Indicates Poor Air Quality
	Cathode (-)	GND	Direct to GND	–

3. POWER SUPPLY

Component	Pin	Connection	Function
ESP8266 NodeMCU	VIN	5V DC (USB / External 5V Supply)	Power Supply to ESP8266
ESP8266 NodeMCU	GND	Common Ground	Ground

6. Circuit Design:



7. Algorithm:

- Start the system.
- Initialize the ESP8266 and connected sensors.
- Connect the ESP8266 to Wi-Fi.
- Establish connection with the cloud platform.
- Read generator temperature.
- Read vibration level.
- Read generator voltage.
- Read generator current.
- Read fuel level and operating status.
- Compare each parameter with its predefined safe limit.
- If all parameters are within acceptable limits, indicate Normal Operation.
- If any parameter exceeds its limit, generate a warning.
- Upload the sensor readings to the cloud.
- Store the measurements for future analysis.
- Generate maintenance recommendations when abnormal conditions or scheduled service intervals are detected.
- Repeat the monitoring process continuously

8. Coding:

```
#include <WiFi.h>

#include "AdafruitIO_WiFi.h"

#include <Wire.h>

#include <hd44780.h>

#include <hd44780ioClass/hd44780_I2Cexp.h>

#include <DHT.h>

// =====

// WIFI

// =====
```

```

#define WIFI_SSID "OnePlus Nord CE5 7C5C"

#define WIFI_PASS "qkts5737"

// =====

// ADAFRUIT IO

// =====

#define IO_USERNAME "Sivasaravanan"

#define IO_KEY    "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"

// Adafruit IO connection

AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASS
)

// =====

// ADAFRUIT IO FEEDS

// =====

AdafruitIO_Feed *currentFeed =
  io.feed("generator-current");

AdafruitIO_Feed *temperatureFeed =
  io.feed("generator-temperature")

AdafruitIO_Feed *humidityFeed =
  io.feed("generator-humidity");

AdafruitIO_Feed *statusFeed =
  io.feed("generator-status");

// =====

// LCD

// =====

hd44780_I2Cexp lcd;

```

```

// =====
// DHT11
// =====
#define DHT_PIN 14
#define DHT_TYPE DHT11
DHT dht(DHT_PIN, DHT_TYPE);
// =====
// ACS712
// =====
#define ACS712_PIN 34
/*
  ACS712 sensitivity
  5A  = 0.185 V/A
  20A = 0.100 V/A
  30A = 0.066 V/A
  This code currently uses ACS712 20A.
*/
float ACS_SENSITIVITY = 0.100;
// Zero-current voltage
float ZERO_CURRENT_VOLTAGE = 2.50;
// =====
// BUZZER
// =====
#define BUZZER_PIN 27
// =====
// WARNING LIMITS
// =====
float MAX_CURRENT = 20.0;
float MAX_TEMPERATURE = 80.0;

```



```
// =====  
// VARIABLES  
// =====  
  
float currentValue = 0.0;  
  
float temperatureValue = 0.0;  
  
float humidityValue = 0.0;  
  
bool warningState = false;  
  
// =====  
// SETUP  
// =====  
  
void setup()  
{  
  Serial.begin(115200)  
  
  delay(1000);  
  
  Serial.println();  
  
  Serial.println("=====");  
  
  Serial.println(" INDUSTRIAL DIESEL GENERATOR MONITOR ");  
  
  Serial.println("=====");  
  
  // -----  
  
  // BUZZER  
  
  // -----  
  
  pinMode(BUZZER_PIN, OUTPUT);  
  
  digitalWrite(BUZZER_PIN, LOW);  
  
  // -----  
  
  // DHT11  
  
  // -----  
  
  dht.begin();  
  
  // -----  
  
  // I2C LCD
```

```
//  
// ESP32:  
// SDA = GPIO 21  
// SCL = GPIO 22  
// -----  
Wire.begin(21, 22);  
// -----  
// LCD INITIALIZATION  
// -----  
int lcdStatus = lcd.begin(16, 2);  
if (lcdStatus != 0)  
{  
    Serial.print("LCD ERROR: ");  
    Serial.println(lcdStatus);  
    while (true)  
    {  
        delay(100);  
    }  
}  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("DIESEL GENERATOR");  
lcd.setCursor(0, 1);  
lcd.print("MONITORING");  
delay(2500);  
// -----  
// ADAFRUIT IO  
// -----  
lcd.clear();
```

```

lcd.setCursor(0, 0);
lcd.print("Connecting...");
Serial.println("Connecting to Adafruit IO...");
io.connect();

// Wait for Adafruit IO
while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("ADA IO CONNECTED");
lcd.setCursor(0, 1);
lcd.print("SYSTEM READY");
delay(2000);
}

// =====
// READ ACS712
// =====

float readCurrent()
{
  const int samples = 500;
  long totalADC = 0;
  for (int i = 0; i < samples; i++)
  {
    totalADC += analogRead(ACS712_PIN);
  }
}

```

```

delayMicroseconds(100);
}
// Average ADC value
float averageADC =
    (float)totalADC / samples;
// Convert ADC to voltage
float sensorVoltage =
    averageADC * 3.3 / 4095.0;
// Calculate current
float current =
    (sensorVoltage - ZERO_CURRENT_VOLTAGE)
    / ACS_SENSITIVITY;
// Remove small noise
if (abs(current) < 0.20)
{
    current = 0;
}
return abs(current);
}

// =====
// CHECK WARNING
// =====

void checkWarning()
{
    warningState = false;
    // High current
    if (currentValue >= MAX_CURRENT)
    {
        warningState = true;
    }
}

```

```
}  
  
// High temperature  
if (temperatureValue >= MAX_TEMPERATURE)  
{  
    warningState = true;  
}  
  
// Buzzer  
if (warningState)  
{  
    digitalWrite(BUZZER_PIN, HIGH);  
}  
else  
{  
    digitalWrite(BUZZER_PIN, LOW);  
}  
}  
  
// =====  
// LCD SCREEN 1  
// =====
```

```
void displayScreen1()  
{  
    lcd.clear()  
    lcd.setCursor(0, 0);  
    lcd.print("CURRENT:");  
    lcd.print(currentValue, 1);  
    lcd.print("A");  
    lcd.setCursor(0, 1);  
}
```

```
lcd.print("TEMP:");  
lcd.print(temperatureValue, 1);  
lcd.print("C");  
delay(2500);  
}  
// =====  
// LCD SCREEN 2  
// =====  
void displayScreen2()  
{  
  lcd.clear();  
  lcd.setCursor(0, 0);  
  lcd.print("HUMIDITY:");  
  lcd.print(humidityValue, 0);  
  lcd.print("%");  
  lcd.setCursor(0, 1);  
  if (warningState)  
  {  
    lcd.print("STATUS: WARNING");  
  }  
  else  
  {  
    lcd.print("STATUS: NORMAL");  
  }  
  delay(2500);  
}  
// =====  
// SERIAL MONITOR  
// =====
```

```

void printData()
{
    Serial.println();
    Serial.println("-----");
    Serial.print("Current  : ");
    Serial.print(currentValue, 2);
    Serial.println(" A");
    Serial.print("Temperature : ");
    Serial.print(temperatureValue, 2);
    Serial.println(" C");
    Serial.print("Humidity   : ");
    Serial.print(humidityValue, 2);
    Serial.println(" %");
    if (warningState)
    {
        Serial.println("Status    : WARNING");
    }
    else
    {
        Serial.println("Status    : NORMAL");
    }
    Serial.println("-----");

    // =====
    // SEND DATA TO ADAFRUIT IO
    // =====

    void sendData()
    {
        Serial.println("Sending data to Adafruit IO...")
    }
}

```

```

// Current
currentFeed->save(currentValue);
delay(300);

// Temperature
temperatureFeed->save(temperatureValue);
delay(300);

// Humidity
humidityFeed->save(humidityValue);
delay(300);

// Status
if (warningState)
{
    statusFeed->save("WARNING");
}
else
{
    statusFeed->save("NORMAL");
}

Serial.println("Data sent successfully!");
}

// =====

// LOOP

// =====

void loop()
{
    // Keep Adafruit IO connected
    io.run();

    // -----

    // DHT11

```



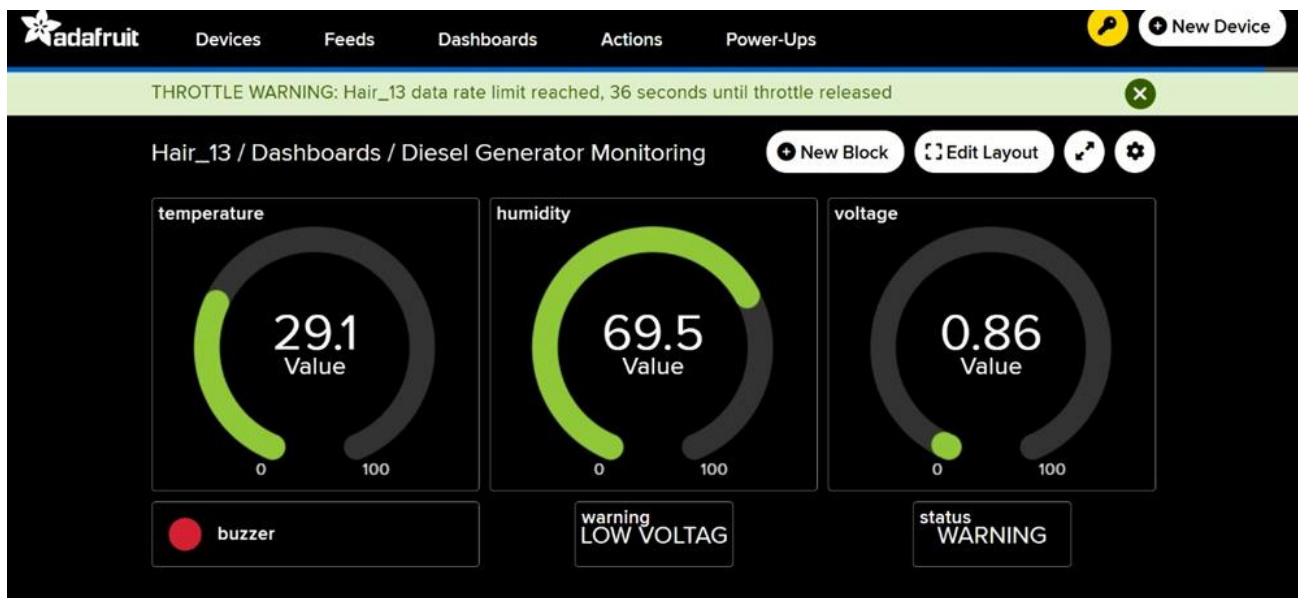
```
// -----  
humidityValue = dht.readHumidity();  
temperatureValue = dht.readTemperature();  
// Check DHT11  
if (isnan(humidityValue) ||  
    isnan(temperatureValue))  
{  
    Serial.println("ERROR: DHT11 reading failed!");  
    delay(2000);  
    return;  
}  
// -----  
// ACS712  
// -----  
currentValue = readCurrent();  
// -----  
// WARNING  
// -----  
checkWarning();  
// -----  
// SERIAL MONITOR  
// -----  
printData();  
// -----  
// ADAFRUIT IO  
// -----  
sendData();  
// -----  
// LCD
```

```

// -----
displayScreen1();
displayScreen2();
// -----
// WAIT
// -----
delay(2000);
}

```

9. System Working :



The Industrial Diesel Generator Monitoring and Maintenance System works by continuously monitoring the important operating parameters of a diesel generator using different sensors. An ESP8266 Node MCU acts as the main controller and collects the sensor information.

When the generator is switched ON, the sensors start measuring parameters such as engine temperature, vibration, generator voltage, current,

fuel level, and operating status. The sensor outputs are sent to the ESP8266 for processing.

The ESP8266 compares the measured values with predefined safe operating limits. If the parameters are within the normal range, the generator is considered to be operating normally. If any parameter exceeds its safe limit, the system identifies an abnormal condition and generates a warning for maintenance.

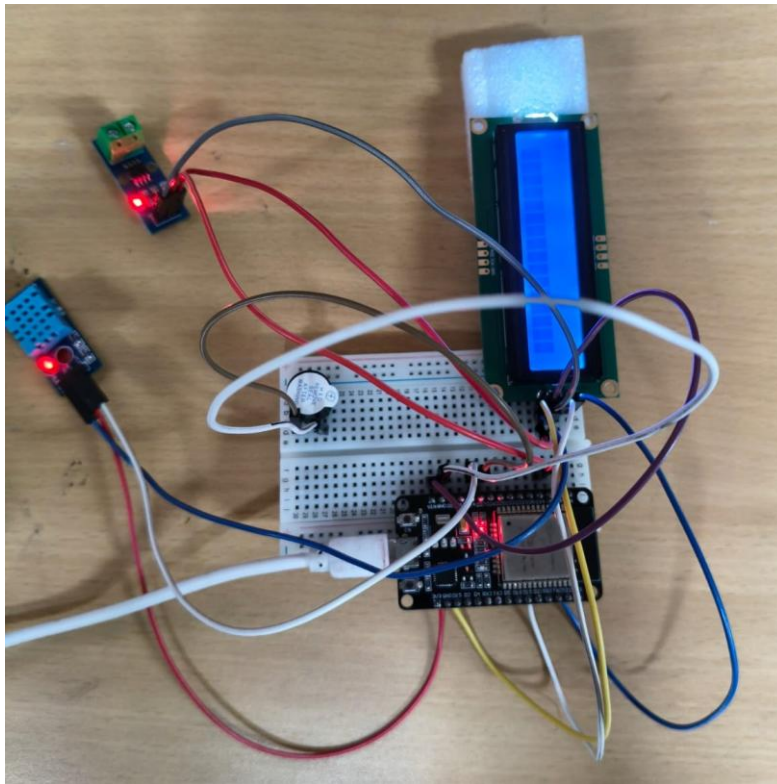
The ESP8266 is connected to the Internet through Wi-Fi and sends the collected data to a cloud platform. The cloud dashboard displays the generator parameters in real time and stores the data for future analysis.

10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component	Specification / Description	Quantity
1	ESP8266 NodeMCU	Wi-Fi Development Board	1
2	DS18B20 Temperature Sensor	Digital Temperature Sensor	1
3	Fuel Level Sensor	Resistive / Ultrasonic Type	1
4	Vibration Sensor	SW-420 / Analog Type	1
5	Battery Voltage Sensor	Voltage Divider Module	1
6	LED	5mm LED	1
7	Resistor	220 Ω	1
8	Breadboard / PCB	Prototype Board	1
9	Jumper Wires	Male to Male / Male to Female	As required
10	USB Cable	Micro USB	1
11	Wi-Fi Router	Internet Connection	1
12	Adafruit IO	Cloud Platform	1

11. Prototype Implementation:



The prototype of the Industrial Diesel Generator Monitoring and Maintenance System was developed using an ESP8266 NodeMCU as the main control unit. The ESP8266 collects data from sensors connected to the generator and processes the measured parameters to determine its operating condition.

The prototype includes sensors for monitoring important generator parameters such as engine temperature, vibration, voltage, current, and fuel level. The sensors are connected to the appropriate input pins of the ESP8266. The controller continuously reads the sensor values and compares them with predefined threshold limits.

The ESP8266 is programmed using the Arduino IDE. The program is designed to identify normal and abnormal operating conditions. When all parameters remain within the safe range, the generator status is displayed as

Normal. If any parameter exceeds its predefined limit, the system generates a Warning and indicates that inspection or maintenance may be required.

The ESP8266 is connected to a Wi-Fi network. The measured generator parameters are transmitted to a cloud platform and displayed on a monitoring dashboard. The dashboard allows the operator to observe the generator condition remotely and view stored sensor data.

The prototype can display parameters such as:

- Engine temperature
- Generator voltage
- Generator current
- Vibration level
- Fuel level
- Generator ON/OFF status
- Maintenance or warning status

The implemented prototype demonstrates the feasibility of using IoT technology for industrial diesel generator monitoring and preventive maintenance. It provides real-time monitoring, remote data access, abnormal-condition detection, and maintenance support, which can help reduce unexpected generator failures and improve maintenance planning.

12. Results and Performance Analysis:

The developed Industrial Diesel Generator Monitoring and Maintenance System was successfully implemented as an IoT-based prototype. The system was tested to verify its ability to monitor important generator parameters and identify abnormal operating conditions.

During testing, the sensors provided data related to engine temperature, vibration, generator voltage, current, fuel level, and operating

status. The ESP8266 NodeMCU successfully collected and processed the sensor data. The measured values were compared with predefined threshold limits to determine whether the generator was operating normally or required attention.

When the monitored parameters remained within the specified limits, the system indicated Normal Operation. When a parameter exceeded its threshold, the system generated a warning condition and indicated that maintenance or inspection may be required.

The ESP8266 successfully transmitted the collected data through Wi-Fi to the cloud platform. The data could be displayed on a dashboard, allowing the operator to monitor the generator remotely and observe changes in its operating condition.

The system provides real-time monitoring and remote access to generator operating information. Cloud-based data storage also allows historical readings to be analyzed to identify changes in generator performance.

The system can help reduce dependence on manual inspection and support preventive maintenance. Early detection of high temperature, excessive vibration, abnormal voltage/current, or low fuel level can help operators take corrective action before a major failure occurs.

The prototype demonstrates that an IoT-based monitoring system can be effectively used for diesel generator performance monitoring and preventive maintenance support. For actual industrial deployment, the sensors should be properly calibrated and suitable electrical isolation and protection should be used for measuring generator voltage and current.